

Gra w kafelki

Barchin i Charos grają w grę na planszy złożonej z $2N \times 2M$ kwadratów jednostkowych. Wiersze są ponumerowane od 0 do $2N - 1$ od góry do dołu, a kolumny są ponumerowane od 0 do $2M - 1$ od lewej do prawej. Dla $0 \leq i < 2N$ i $0 \leq j < 2M$ oznaczamy komórkę w wierszu i i kolumnie j przez (i, j) .

Barchin daje Charosowi $N \cdot M$ **bloków**, jeden po drugim. Każdy blok to kwadrat 2×2 składający się z czterech **kafelków** 1×1 . Barchin pokolorował każdy kafelek bloku na czarno lub biało, gwarantując jednocześnie, że **przynajmniej jeden kafelek będzie biały**.

Charos musi umieścić każdy blok na planszy **natychmiast po jego otrzymaniu**, nie wiedząc, jakie bloki otrzyma później. Bloków nie można obracać. Każdy blok musi być umieszczony w całości wewnątrz planszy, pokrywając dokładnie cztery pola. Ponadto, kafelek w lewym górnym rogu każdego bloku musi pokrywać pole, którego współrzędne wiersza i kolumny są parzyste. Każda komórka planszy może być pokryta co najwyżej jednym blokiem.

Barchin wygrywa grę, jeśli po umieszczeniu dowolnego bloku istnieje kwadrat złożony z 2×2 komórek, który jest pokryty czterema czarnymi kafelkami. Formalnie, jeśli komórki (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ są pokryte czarnymi kafelkami dla pewnego $0 \leq a < 2N - 1$ i $0 \leq b < 2M - 1$, to Barchin wygrywa. Indeksy a i b nie muszą być parzyste.

Charos wygrywa, jeśli umieści wszystkie $N \cdot M$ bloków, zanim Barchin wygra. Zauważ, że umieszczenie $N \cdot M$ bloków spowoduje całkowite pokrycie planszy.

Twoim zadaniem jest zaimplementowanie strategii, która umożliwi Charosowi wygranie gry. Można udowodnić, że przy danych ograniczeniach Charos zawsze może rozmieścić bloki w taki sposób, aby zagwarantować sobie wygraną, bez względu na kolory bloków, które otrzyma później.

Szczegóły implementacji

Powinieneś zaimplementować dwie procedury:

```
void init(int N, int M)
```

- N : połowa liczby wierszy planszy.
- M : połowa liczby kolumn planszy.

- Procedura jest wywoływana dokładnie raz dla każdego przypadku testowego na początku wykonywania programu.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : kolory odpowiednio kafelków w lewym górnym rogu, prawym górnym rogu, lewym dolnym rogu i prawym dolnym rogu bieżącego bloku, jak pokazano na poniższym rysunku. Każda wartość to 0 (biały) lub 1 (czarny).
- Ta procedura jest wywoływana dokładnie $N \cdot M$ razy na przypadek testowy po początkowym wywołaniu `init`.



Ta procedura powinna zwrócić parę liczb całkowitych (i, j) , gdzie i jest współrzędną wiersza, a j jest współrzędną kolumny komórki, w której powinien zostać umieszczony lewy górny kafelek tego bloku. Zarówno i jak i j muszą być parzyste, a obszar 2×2 pokryty blokiem nie może nachodzić na żaden wcześniej umieszczony blok.

Jeżeli `receive_block` zwróci parę, która nie spełnia tych wymagań, lub jeżeli po umieszczeniu bloku pewien kwadrat komórek o wymiarach 2×2 zostanie całkowicie pokryty czarnymi kafelkami, program sprawdzający natychmiast zakończy działanie programu, a werdykt dotyczący przypadku testowego będzie brzmiał `Output isn't correct`.

Zachowanie programu oceniającego **nie jest adaptacyjne**. Oznacza to, że ciąg bloków przekazywanych przez Barchina do Charosa jest ustalony przed wywołaniem `init`.

Ograniczenia

Dla każdego bloku niech S będzie liczbą czarnych kafelków wśród czterech kafelków w nim zawartych. Innymi słowy, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ dla każdego bloku.

Podzadania

Podzadanie	Punkty	Dodatkowe ograniczenia
1	6	$S = 1$ dla każdego bloku i $N = 2$.
2	16	$S = 3$ dla każdego bloku. $N = M$, N jest parzyste, a każde z czterech możliwych pokolorowań bloku pojawia się dokładnie $\frac{N^2}{4}$ razy.
3	10	$S = 1$ dla każdego bloku.
4	29	$S \leq 2$ dla każdego bloku.
5	39	Brak dodatkowych ograniczeń.

Przykład

Rozważmy grę, w której $N = 1$ i $M = 2$, a zatem plansza ma 2 wiersze i 4 kolumny. Program oceniający najpierw wywoła:

```
init(1, 2)
```

Początkowo wszystkie komórki są puste. Plansza wygląda tak:

	0	1	2	3
0				
1				

Należy umieścić $N \cdot M = 2$ bloków. Załóżmy, że Barchin daje blok z trzema czarnymi kafelkami i jednym białym w prawym górnym rogu. Program oceniający wywołuje:

```
receive_block(1, 0, 1, 1)
```

Charos decyduje się umieścić ten blok po lewej stronie planszy zwracając $(0, 0)$.

Plansza wygląda teraz tak:

	0	1	2	3
0				
1				

Następnie Barchin daje kolejny blok z trzema czarnymi kafelkami i jednym białym kafelkiem w lewym górnym rogu:

```
receive_block(0, 1, 1, 1)
```

Jedyną pozostałą komórką z parzystym wierszem i parzystą kolumną, która może służyć jako lewy górny róg bloku 2×2 jest $(0, 2)$, więc Charos zwraca $(0, 2)$. Plansza wygląda następująco:

	0	1	2	3
0				
1				

Żaden kwadrat 2×2 nie jest całkowicie pokryty czarnymi kafelkami, więc Charosowi udało się umieścić wszystkie bloki bez wygranej Barchina. Charos wygrywa grę.

Przykładowy program oceniający

Format wejścia:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Format wyjścia:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Wartości $R[k]$ i $C[k]$ to para liczb całkowitych zwracanych przez k -te wywołanie `receive_block`.